

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 8
з курсу “Інформаційна безпека програм та даних”
на тему: “Алгоритми хешування”

Виконав:

Студент. гр. ФЕП-41 Фіняк Олег

Перевірив:

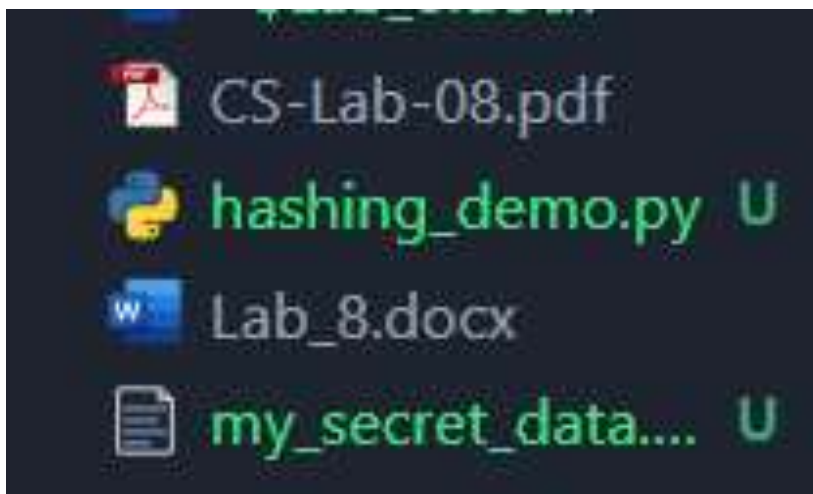
Павлик М.Р.

Львів – 2026

Мета: створити простий сценарій Python для хешування даних

Завдання:

1. Імпортуйте hashlib - бібліотеку Python для хешування:
2. Створіть хеш повідомлення використовуючи алгоритм MD5:
3. Алгоритм MD5 застарів, тому його не слід використовувати, оскільки він не стійкий до злому. Використаємо SHA-2: SHA-2 - це сімейство з 4-х хеш-функцій: SHA-224, SHA-256, SHA-384 і SHA-512, ви також можете використовувати hashlib.sha224() і hashlib.sha384(). Проте в основному використовуються SHA-256 і SHA-512. Причина, чому він називається SHA-2 (Secure Hash Algorithm 2), полягає в тому, що SHA-2 є наступником SHA-1, який застарів і його легко зламати. SHA-2 генерує довші хеші, що призводить до вищого рівня безпеки, ніж SHA-1. Незважаючи на те, що SHA-2 все ще використовується в наш час, багато хто вважає, що атаки на SHA-2 лише питання часу, дослідники стурбовані його довгостроковою безпекою через його схожість із SHA-1.
4. Використаємо SHA-3 - більш безпечний і повністю відмінний алгоритм хешування ніж SHA-2. Існує небагато стимулів для оновлення до SHA-3, оскільки SHA-2 все ще безпечний, а оскільки швидкість також є проблемою (SHA-3 не швидший за SHA-2).
5. Якщо ми хочемо використовувати швидшу хеш-функцію, більш безпечну, ніж SHA-2, і принаймні таку ж безпечну, як SHA-3 то потрібно використовувати BLAKE2. Хешування алгоритмом BLAKE2 швидше, ніж SHA-1, SHA-2, SHA-3 і навіть MD5, і навіть безпечніше, ніж SHA-2. Він підходить для використання на сучасних процесорах, які підтримують паралельні обчислення в багатоядерних системах. BLAKE2 широко використовується та був інтегрований у основні криптографічні бібліотеки, такі як OpenSSL та Sodium тощо.
6. Створіть файл та за хешуйте його, просто прочитайте весь вміст файлу, а потім передайте байти файлу функціям хешування, які ми розглянули. Порівняйте результати роботи цих функцій.



```
PS C:\Users\Finik\Desktop\uni_labs_4\sec_sem\security\Lab_8> & 'c:\Users\Finik\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'c:\Users\Finik\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '64191' '--' 'C:\Users\Finik\Desktop\uni_labs_4\sec_sem\security\Lab_8\hashing_demo.py'
```

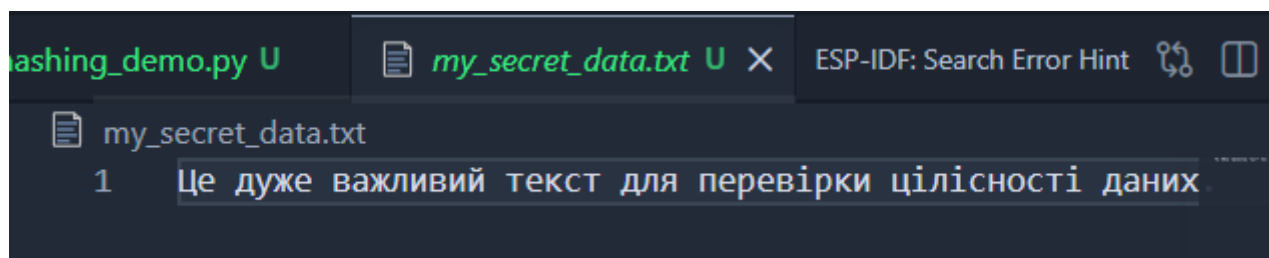
--- 1. Хешування текстового рядка ---

```
MD5: 3eecc85e6440899b28a9ea6d8369f01c
SHA-256: 7a86e0e93e6aa6cf49f19368ca7242e24640a988ac8e5508dfcedec9fa53faa2
SHA-512: 96fa772f72678c85bbd5d23b66d51d50f8f9824a0aba0ded624ab61fe8b602bf4e3611075fe13595d3e74c63c59f7d79241acc97888e9a7a5c791159c85c3ccd
SHA-3-256: d7007c1cd52f8168f22fa25ef011a5b3644bcb437efa46de34761d3340187609
SHA-3-512: de6b4c8f7d4fd608987c123122bcc63081372d09b4bc14955bfc828335dec1246b5c6633c5b1c87d2ad2b777d713d7777819263e7ad675a3743bf2a35bc699d0
BLAKE2s: 6889074426b5454d751547cd33ca4c64cd693f86ce69be5c951223f3af845786
BLAKE2b: 13e2ca8f6a282f27b2022dde683490b1085b3e16a98ee77b44b25bc84a0366afe8d70a4aa47dd10e064f1f772573513d64d56e5ef646fb935c040b32f67e5ab2
```

--- 2. Хешування вмісту файлу 'my_secret_data.txt' ---

```
MD5: 25c62ff0ef604db4c0b2daebbcc546c9
SHA-256: 3a56041b094ff8cecfa6b835c30ba911da60a8b5ef6395a7acb492ed5ac7a7e8
BLAKE2b: 3fe3d8b67de20c4215bf35eb3748992f7fbed53df162b69ba572e32d14787841d45074d3625d3b3c288cffe2df0844feb788ac691671eca18c95dc79fcdd9c14
```

```
PS C:\Users\Finik\Desktop\uni_labs_4\sec_sem\security\Lab_8>
```



Hashing_demo.py

```
import hashlib #

def hash_text_demo():
    print("--- 1. Хешування текстового рядка ---")
    # Перетворюємо рядок у байти за допомогою кодування UTF-8 #
    message = "Some text to hash".encode() #

    # MD5 (Застарілий, не рекомендується використовувати) #
    print(f"MD5: {hashlib.md5(message).hexdigest()}") #

    # SHA-2 (SHA-256 та SHA-512 - надійні, але повільніші) #
```

```

print(f"SHA-256:      {hashlib.sha256(message).hexdigest()}") #
print(f"SHA-512:      {hashlib.sha512(message).hexdigest()}") #

# SHA-3 (Більш безпечний, повністю відмінний від SHA-2) #[cite: 8]
print(f"SHA-3-256:    {hashlib.sha3_256(message).hexdigest()}") #[cite: 8]
print(f"SHA-3-512:    {hashlib.sha3_512(message).hexdigest()}") #[cite: 8]

# BLAKE2 (Швидший за MD5/SHA і безпечніший за SHA-2) #[cite: 8]
print(f"BLAKE2s:      {hashlib.blake2s(message).hexdigest()}") #[cite: 8]
print(f"BLAKE2b:      {hashlib.blake2b(message).hexdigest()}") #[cite: 8]

def hash_file_demo(filename):
    print(f"\n--- 2. Хешування вмісту файлу '{filename}' ---")
    try:
        # Щоб захешувати файл, достатньо прочитати його в бінарному режимі ('rb')
        #[cite: 8]
        with open(filename, 'rb') as f:
            file_bytes = f.read() #[cite: 8]

            print(f"MD5:          {hashlib.md5(file_bytes).hexdigest()}")
            print(f"SHA-256:       {hashlib.sha256(file_bytes).hexdigest()}")
            print(f"BLAKE2b:        {hashlib.blake2b(file_bytes).hexdigest()}")

    except FileNotFoundError:
        print(f"✗ Помилка: Файл '{filename}' не знайдено.")

if __name__ == "__main__":
    # Демонстрація хешування звичайного тексту
    hash_text_demo()

    # Виконання пункту 6: створюємо тестовий файл
    test_file = "my_secret_data.txt"
    with open(test_file, "w", encoding="utf-8") as file:
        file.write("Це дуже важливий текст для перевірки цілісності даних.")

    # Хешуємо щойно створений файл
    hash_file_demo(test_file)

```

Висновок: Під час виконання лабораторної роботи було досліджено принципи роботи криптографічних хеш-функцій та реалізовано скрипт для хешування текстових даних і файлів за допомогою бібліотеки `hashlib`[cite: 8]. Практичне порівняння алгоритмів MD5, SHA-2, SHA-3 та BLAKE2 підтвердило відмінності у довжині їхніх вихідних значень і дозволило засвоїти сучасні підходи до перевірки цілісності даних[cite: 8].